Solving Poker

The numerical analysis behind poker solvers
— and a second-order finish for them

Numerical Analysis Research Club

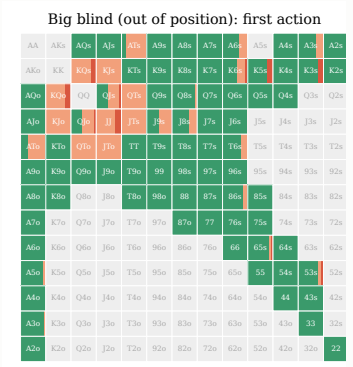
Ten years of superhuman poker

In 2015 a program called Cepheus essentially solved two-player limit hold'em.

In 2017 two programs, DeepStack and Libratus, beat professional players at the no-limit game, which is far larger. In 2019 Pluribus did the same at a table of six.

Today every serious player studies with a solver. You describe one situation, and the program computes how to play every possible hand in it.

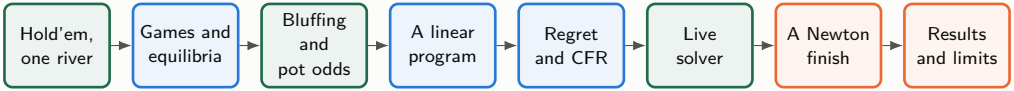
All of these programs share one simple iteration. This talk explains it and then tries to improve on it.



A solver's answer, computed by our own code.

Each square is a starting hand and the colours are actions.

The plan: poker $\leftrightarrow$ math



The equilibrium of a poker game is the solution of a very large and very structured saddle-point problem. Solvers attack it with a first-order method whose steps are cheap and whose last digits are slow.

In the middle of the talk we solve a real hand live in the browser and look at the answer together.

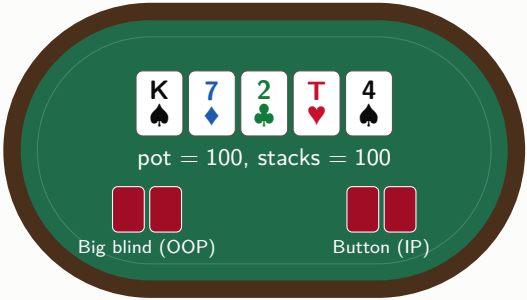
Our own contribution comes last. We treat the end of the solve as a nonsmooth equation and finish it with a Newton method, which reaches machine precision where the first-order method would need millions of iterations.

Texas hold'em in one slide

Each of the two players is dealt two private cards. Five public cards, the *board*, are then revealed in three stages called the flop, the turn and the river.

There is a round of betting before the flop and after each stage. A player may check, bet, call, raise or fold, and in the no-limit game a bet can be any size up to the player's stack.

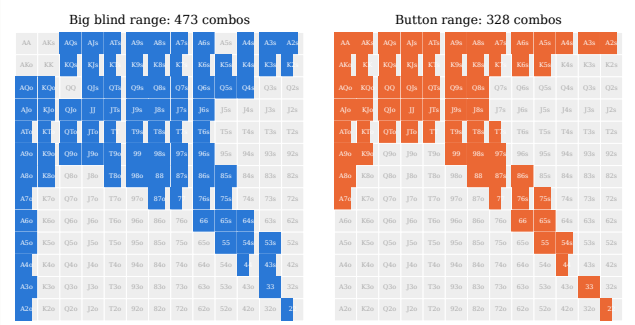
If nobody folds, the best five-card hand made from the seven available cards wins the pot.



Our running example is the river, the last round of betting.

The big blind acts first, which is called being out of position.

Ranges: what a solver takes as input



A **range** is a probability vector over the two-card hands a player can hold. There are 1326 such hands, and 1081 once the five board cards are known.

In our example the big blind called a raise before the flop and the button made that raise, so the two ranges are different.

The solver returns, for every point of the betting and for every hand, a probability for each action.

Two-player zero-sum games and Nash equilibrium

Each player chooses a strategy from a compact convex set, $x \in X$ and $y \in Y$, and the first player receives the bilinear payoff $x^\top A y$ from the second.

Minimax theorem (von Neumann 1928)

$$\max_{x \in X} \min_{y \in Y} x^\top A y \;=\; \min_{y \in Y} \max_{x \in X} x^\top A y \;=:\; v^*.$$

A pair (x^*, y^*) that attains both sides is a **Nash equilibrium**, and v^* is the value of the game. Playing x^* guarantees at least v^* against any opponent, which is what poker players mean by an unexploitable strategy.

The error that every solver reports is the exploitability

$$\mathrm{expl}(x, y) \;=\; \tfrac{1}{2}\Big(\max_{x'} x'^\top A y \;-\; \min_{y'} x^\top A y' \Big) \;\geq 0, \qquad \text{with equality exactly at an equilibrium.}$$

Why equilibria mix: bluffing and pot odds

Take a toy river with a pot of 1. The bettor holds either the best hand or nothing and bets s. The caller holds a medium hand that only beats a bluff.

The caller is indifferent between calling and folding when bluffs are a fraction $\beta = \dfrac{s}{1+2s}$ of the bets.

The bettor's bluffs are indifferent when the caller continues with probability $\dfrac{1}{1+s}$. Poker players call this the *minimum defence frequency*.

Any pure strategy can be exploited here. The equilibrium randomises, and it does so exactly where the opponent is made indifferent.

bet s	bluffs β	defend
1/3	20%	75%
3/4	30%	57%
1	33%	50%
2	40%	33%

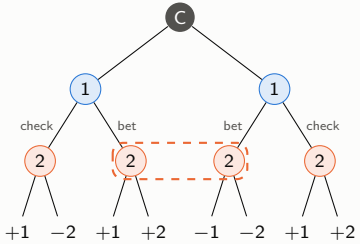
We will check the last column against the live solver.

Games on trees with hidden information

Poker is a game tree. Chance deals the cards, the players act in turn, and the leaves carry the payoffs.

A player cannot see the opponent's cards, so several nodes look the same to that player. Such a group of nodes is an **information set**, and a strategy assigns a probability distribution over actions to each one.

game	information sets
Kuhn poker	12
Leduc hold'em	288
Limit hold'em	3.2×10^{14}
No-limit hold'em	about 10^{161}



Player 2 sees that player 1 bet but not which card was dealt, so the two boxed nodes are one information set.

The sequence form: equilibrium is a linear program

Give player 1 one variable x_σ for each *sequence* σ of her own actions, equal to the product of her probabilities along it. These variables satisfy linear flow constraints $Ex = e$, $x \geq 0$, and the expected payoff is bilinear in them, $u(x, y) = x^\top A y$.

Theorem (Koller, Megiddo and von Stengel 1996)

An equilibrium strategy of player 1 solves

$$\max_{x,\, q} \; f^\top q \quad \text{subject to} \quad A^\top x - F^\top q \geq 0, \quad Ex = e, \quad x \geq 0,$$

a linear program whose size is *linear* in the size of the game tree.

So why not call an LP solver? On our river A has 2.9 million nonzeros, because every leaf couples every hand of one player to every hand of the other, and a good LP code did not finish in 25 minutes. The iteration we describe next reaches 1.3×10^{-4} chips in 16 seconds.

Regret matching

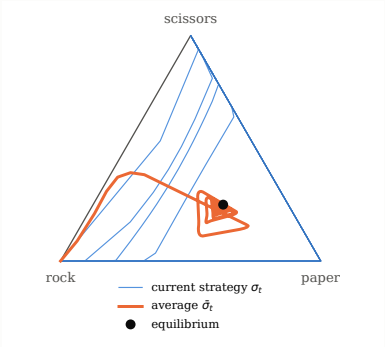
We face the same decision many times. The *regret* $R_T(a)$ of an action is how much better it would have done than what we played. Regret matching plays each action in proportion to its positive regret.

$$R_T(a) = \sum_{t=1}^{T} \big(u_t(a) - u_t(\sigma_t)\big), \quad \sigma_{T+1}(a) = \frac{[R_T(a)]^+}{\sum_b [R_T(b)]^+}.$$

Theorem (Hart and Mas-Colell 2000)

The regrets grow only like $\sqrt{T}$:
$\max_a R_T(a) \le \Delta \sqrt{|A|\ T}$.

When both players of a zero-sum game do this, their *average* strategies converge to an equilibrium.



Our computation on a weighted rock-paper-scissors game. The current strategy cycles and the average converges.

Counterfactual regret minimisation (CFR)

CFR runs regret matching separately at every information set I. The payoff it feeds to each one is the **counterfactual value** of an action, which is the expected payoff weighted by the probability that the opponent and chance lead to I, as if the player herself had tried to get there.

$$v_i(\sigma, I, a) = \sum_{h \in I} \pi^\sigma_{-i}(h) \sum_{z \sqsupset ha} \pi^\sigma(ha \to z)\, u_i(z)$$

Theorem (Zinkevich, Johanson, Bowling and Piccione 2007)

The regret of the whole strategy is at most the sum of the local regrets. Therefore the average strategies satisfy

$$\mathrm{expl}(\bar{x}_T, \bar{y}_T) \;\le\; \frac{\Delta\, |\mathcal{I}| \sqrt{|A|}}{\sqrt{T}}.$$

One iteration is one pass over the tree, with reach probabilities computed on the way down and values on the way up. The matrix A is never formed.

CFR for poker: one pass for all hands

In poker the betting is public and only the cards are hidden. At each point of the betting we store *vectors* indexed by hands, such as the reach probabilities π_1 and π_2. At a leaf the values of all hands come from one matrix-vector product.

$$v_1 = \text{amount} \cdot B\,\pi_2, \qquad B_{ij} = \underbrace{C_{ij}}_{\text{no shared card}} \cdot \begin{cases} 1 & \text{after a fold} \\ \operatorname{sign}(s_i - s_j) & \text{at a showdown} \end{cases}$$

Here s_i is the strength of hand i. The matrix B is dense but very structured. Once the hands are sorted by strength, the sign matrix is all ones below the diagonal and all minus ones above it, and C is the all-ones matrix minus a sparse correction for hands that share a card.

As a result $B\pi$ costs one sort and a running sum, which is $O(n \log n)$ operations and not n^2.

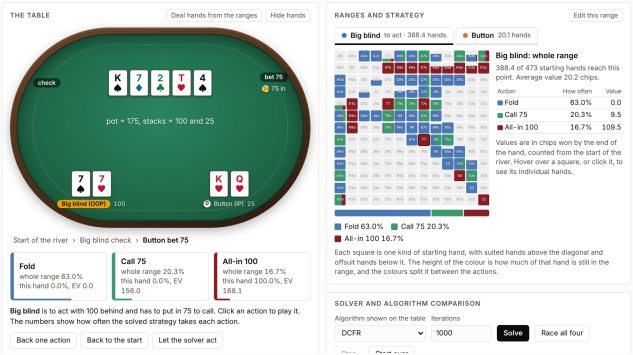
Four variants of one loop

Modern solvers change how CFR weights the old regrets and how it averages the strategies.

method	what changes	a numerical analyst would say
CFR$^+$	Negative regrets are reset to zero and later strategies get more weight in the average.	projection onto the nonnegative orthant
DCFR	Old regrets and old strategies are multiplied by factors below one.	forgetting factors that damp the transient
PCFR$^+$	The strategy is computed from the regrets plus a prediction of the next regret.	optimism, as in the extragradient method

The proven rate is still $T^{-1/2}$ for all four. In practice the modern variants are much faster, and we will now race them against each other on a real hand.

Live demonstration: the river solver table



The big blind's answer to a three-quarter-pot bet. Each square is a starting hand; blue folds, green calls, red raises all-in.

The page `app/poker-solver.html` solves our river in the browser. It will answer five questions.

1. Which of the four algorithms wins a race?
2. What does the big blind do with each of its 473 hands?
3. How do the ranges and the tree change as the hand is played?
4. Does a hand that mixes get the same value from both actions?
5. Does the solver agree with the pot odds formula?

After that the board, the ranges and the bet sizes are yours to change.

What the demonstration showed

The solution reads like poker. Strong hands bet, a few hands with nothing bluff, and the hands in between check.

A hand that mixes two actions gets the same value from both. This is complementary slackness.

Against the big blind's bets the button defends within 1.5 points of the pot odds formula. Against the button's bets and against all-in raises the defender folds much more, because there the bluffs give up something by betting.

defender	line	s	defends	MDF
Button	BB checks, BTN bets 75, BB jams	0.10	64.7%	90.9%
Button	BB bets 33	0.33	74.3%	75.2%
Button	BB bets 75	0.75	55.8%	57.1%
Button	BB bets 100	1.00	49.1%	50.0%
Big blind	BB bets 75, BTN jams	0.10	63.4%	90.9%
Big blind	BB checks, BTN bets 33	0.33	52.2%	75.2%
Big blind	BB bets 33, BTN jams	0.40	56.3%	71.2%
Big blind	BB checks, BTN bets 75	0.75	37.5%	57.1%
Big blind	BB checks, BTN bets 100	1.00	32.6%	50.0%

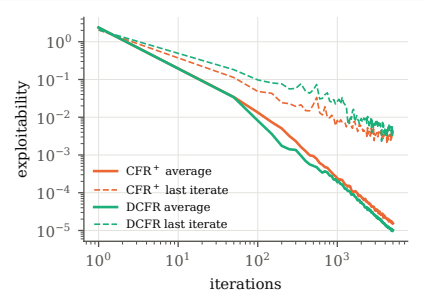
Exact equilibrium of our river. Here s is the bet as a fraction of the pot, MDF is $1/(1+s)$, and a jam is an all-in bet.

Why push the precision further?

Practical solvers stop at about 0.1% of the pot, and for a human studying a hand that is plenty.

More digits matter when a program stitches many small solves together and their errors add up, when we ask exactly which hands mix, and when research needs a ground truth with a certificate.

For a numerical analyst, CFR is a first-order method on a nonsmooth and degenerate problem. It is robust and cheap, and its tail is sublinear. The classical cure for a slow tail is a local method of higher order, started from a good point.



Our computation on Leduc. Only the average converges, and each extra digit costs about ten times more iterations.

Step 1: write the equilibrium as a variational inequality

Collect both strategies and both vectors of information-set values in one unknown $z = (x, y, p, q)$. An equilibrium is a point that satisfies the optimality conditions of both players' linear programs.

$$0 \le x \;\perp\; E^\top p - Ay \;\ge 0, \qquad Ex = e,$$
$$0 \le y \;\perp\; A^\top x - F^\top q \;\ge 0, \quad Fy = f.$$

These conditions have the form $w = Mz + b$ with a matrix that is **skew-symmetric**.

$$M = \begin{pmatrix} 0 & -A & E^\top & 0 \\ A^\top & 0 & 0 & -F^\top \\ -E & 0 & 0 & 0 \\ 0 & F & 0 & 0 \end{pmatrix}, \qquad M + M^\top = 0.$$

Skew symmetry makes the problem monotone. It also means that the solutions form a polyhedron and not a single point, so Newton's method cannot be applied directly. CFR gives a starting point for all of z for free.

Step 2: the proximal point method makes each problem well posed

We add a small multiple of the identity. Given z^k, the next iterate z^{k+1} solves the same conditions with

$$w_\rho(z) = (M + \rho I)\, z + b - \rho\, z^k.$$

Since $d^\top (M + \rho I) d = \rho \|d\|^2$, the matrix $M + \rho I$ is positive definite and each regularised problem has exactly one solution.

Theorem (Rockafellar 1976, Luque 1984). The iterates converge to a solution of the original problem for every $\rho > 0$. For a polyhedral problem the rate is linear, with a factor that tends to zero with ρ.

A small ρ gives a fast outer loop and badly conditioned inner problems.

Our computation on Leduc with $\rho = 10^{-5}$

The residual after each outer step:

$3.4 \times 10^{-2} \to 4.8 \times 10^{-6} \to 3.3 \times 10^{-9} \to 6.9 \times 10^{-12} \to 2.0 \times 10^{-14}$

About three digits for each step.

Step 3: Newton's method on a piecewise-linear equation

Each regularised problem can be written as one equation. For every action, either its probability is zero or its reduced cost is zero, and that is the same as $\min\big(x_i,\ w_\rho(z)_i\big) = 0$.

A Newton step for this equation guesses which actions are dropped, imposes indifference on all the others, and solves one linear system. Its matrix has the form $V = D_a + D_b(M + \rho I)$ with nonnegative diagonal matrices D_a and D_b.

Lemma. Every such V with $D_a + D_b$ positive definite is nonsingular.

Suppose $Vd = 0$ and put $u = (M + \rho I)d$. Row i reads $(D_a)_{ii} d_i + (D_b)_{ii} u_i = 0$ with nonnegative coefficients that are not both zero, so $d_i u_i \le 0$. Hence $\rho\|d\|^2 = d^\top u = \sum_i d_i u_i \le 0$, and $d = 0$. □

If the guess of the dropped actions is right, a single step gives the *exact* solution. If it is wrong there is no global guarantee, so the method needs a good starting guess.

Step 4: making it robust and fast

A robust start. The first regularised problem is the hardest. We solve it with an interior-point method started at the CFR point. Its solution tells us which actions are dropped, and from there the Newton steps land exactly on the solution. Every later problem needs only one to three linear solves.

Fast linear algebra. After a change of signs in half of the rows, every linear system is symmetric and *quasi-definite*. By a theorem of Vanderbei, such a matrix has an $LDL^\top$ factorisation for every ordering of the unknowns, so we can order for sparsity alone and never pivot.

One factorisation, in seconds

	small river	our river
general sparse LU	0.11	25
quasi-definite $LDL^\top$	0.04	7.6

The whole solve from the CFR point to a residual of 10^{-13} takes 19 linear solves on Leduc and 17 on the small river.

Step 5: restrict to the support, solve exactly, then verify

On larger hands the method so far fails. Many actions are never played and many positions are never reached, and in those directions the set of equilibria is *flat*. A point with a tiny residual can be far from a solution, and the Newton step guesses the wrong dropped actions.

The cure is to remove those directions. We keep only the actions that DCFR plays with probability above 10^{-3} and solve this smaller problem exactly.

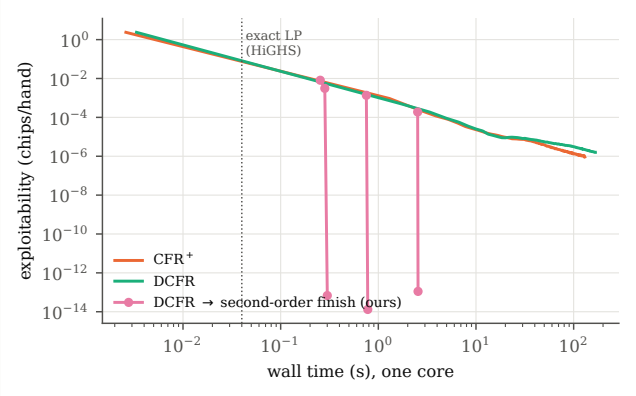
Then we check the answer in the full game. If a removed action would have been profitable we put it back and solve again.

The complete method

1. DCFR for a few thousand iterations
2. keep the actions that it uses
3. solve the restricted problem to machine precision
4. compute exact best responses in the full game
5. add back any profitable action and return to 3

Step 4 is a certificate. The final exploitability is measured in the full game and not assumed.

Result on the benchmark game, Leduc hold'em

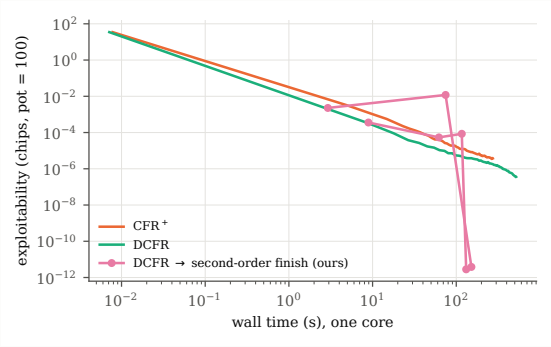


Each pink curve starts where a short DCFR run stopped and drops straight down to machine precision.

From 100 DCFR iterations our method reaches 6.8×10^{-14} in 0.30 seconds. CFR^{+} is at 9.2×10^{-7} after 130 seconds.

Leduc is tiny and an LP solver needs 0.04 seconds. What matters is the shape of the curves.

Result on the river from the demonstration



473 against 328 hands and 17,626 unknowns. The points are verification rounds.

Our method runs DCFR for 3,000 iterations and then needs 3 rounds on a problem with 34% of the unknowns. It reaches 2.8×10^{-12} chips in 131 seconds.

At that moment DCFR alone is at 3.9×10^{-6}, and after 190,000 iterations and 520 seconds it is at 3.6×10^{-7}.

Without the restriction to the support the Newton finish *loses* to DCFR on this hand. The LP solver did not finish in 25 minutes.

Where it stops winning, and what is open

Full ranges: all 1,081 hands for both players, 47,568 unknowns

The restricted problem still has 43% of the unknowns and one factorisation takes about 10 seconds. After 80 minutes our method is at 4.1×10^{-6} chips. DCFR reaches 9.5×10^{-6} in 10 minutes. **Here CFR wins.**

The bottleneck is the linear algebra. Each payoff block is dense, but it is a sorted sign matrix plus a sparse correction, and a product with it costs $O(n \log n)$. A Newton–Krylov step would then cost only a few CFR iterations, provided we can find a good preconditioner for systems of the form "diagonal plus skew". We think this is the most promising open problem, and it is a problem in numerical linear algebra.

Two further directions are a better prediction of the support from the CFR regrets, which would save verification rounds, and the use of exact solves inside programs that play in real time.

What is new here, and what is not

The ingredients are classical: the sequence form, the proximal point method, semismooth Newton, interior point with crossover, quasi-definite factorisations and the CFR family.

The closest work is by Yang, Gao, Lin and Kroer (2025), who use a semismooth Newton method for *matrix* games after a first-order warm start.

What we add is a second-order finish that works directly on the sequence form of a poker game. It has four parts:

- the skew formulation with a proximal term, which gives nonsingular Newton matrices by a one-line proof,
- a warm start for both the strategies and the values taken from CFR,
- the restriction to the support predicted by CFR, which removes the flat directions,
- and the verification by exact best responses in the full game.

Takeaways

A poker equilibrium is the solution of a structured saddle-point problem, and the sequence form turns it into a linear program whose size is linear in the game tree.

CFR is regret matching at every information set. It never forms the matrix, its proven rate is $T^{-1/2}$, and its modern variants are much faster in practice, as the race in the browser showed.

The output is readable poker: value bets, bluffs, traps and a defence that follows the pot odds.

Read as a skew variational inequality, the same problem can be finished by a proximal Newton method. The obstacle is the flatness of the equilibrium set, and restricting to the support that CFR predicts removes it. On our river this gives an exploitability of 10^{-12} in about two minutes, where DCFR alone is at 10^{-6}. With full ranges the linear algebra becomes the bottleneck and CFR wins again.

The solver, the interactive page, the benchmarks and these slides are at `github.com/dwangnumbers/Poker-Solver`.

Main references

- D. Koller, N. Megiddo, B. von Stengel. Efficient computation of equilibria for extensive two-person games. *Games Econ. Behav.* 14 (1996).
- M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret minimization in games with incomplete information. *NeurIPS* 2007.
- N. Brown, T. Sandholm. Solving imperfect-information games via discounted regret minimization. *AAAI* 2019.
- R. T. Rockafellar. Monotone operators and the proximal point algorithm. *SIAM J. Control Optim.* 14 (1976).
- M. Hintermüller, K. Ito, K. Kunisch. The primal-dual active set strategy as a semismooth Newton method. *SIAM J. Optim.* 13 (2002).
- R. J. Vanderbei. Symmetric quasidefinite matrices. *SIAM J. Optim.* 5 (1995).
- D. Yang, Y. Gao, T. Lin, C. Kroer. A direct second-order method for solving two-player zero-sum games. arXiv:2512.12910 (2025).

The complete list of references is in `talk-detailed.pdf`.

Backup: Kuhn poker, where the equilibrium is not unique

Kuhn poker uses the three cards 

. Each player puts in one chip and gets one card, and there is a single bet of one chip.

Its equilibria form a one-parameter family. The first player bets the jack as a bluff with any probability α between 0 and $\frac{1}{3}$, and bets the king with probability 3α. The value of the game is $-\frac{1}{18}$.

The set of equilibria is a polytope and not a point, and any method has to cope with that.

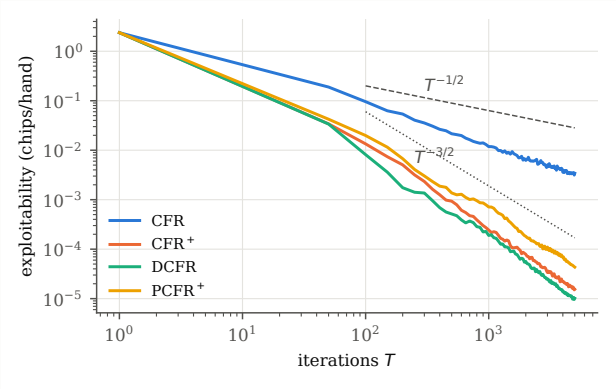
Our computation

The Newton finish returns $\alpha = 0.198$ and bets the king with probability $0.593 = 3\alpha$.

Value $-0.0555\ldots = -1/18$.

Exploitability 1.1×10^{-16}.

Backup: the first-order methods on Leduc hold'em

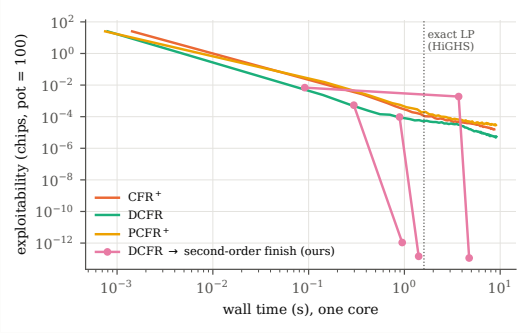


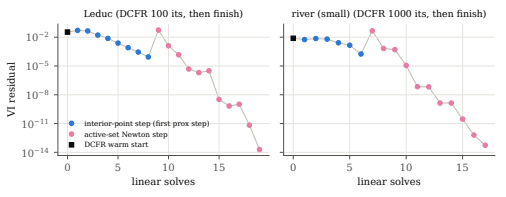
The curves show the exploitability of the average strategy for our implementation.

After 5,000 iterations and 14 seconds DCFR reaches 1.0×10^{-5}. After 50,000 iterations and 130 seconds CFR$^+$ reaches 9.2×10^{-7} and DCFR 1.6×10^{-6}.

Each extra digit costs about ten times more iterations.

Backup: a small river, and every linear solve of one exact solve





On the left is a river with 147 against 139 hands. Our method reaches 1.1×10^{-12} chips in 0.95 seconds, DCFR is at 5.4×10^{-6} after 9 seconds, and the LP takes 1.6 seconds. On the right is the residual after every linear solve. The blue points are interior-point iterations and the pink points are Newton steps.

Backup: the rate of the proximal point method, and why flat equilibria hurt

The proximal step is the resolvent of a maximal monotone operator T, so the iterates move monotonically towards the solution set Z.

If T^{-1} is upper Lipschitz at 0 with modulus a, which holds for polyhedral problems, then

$$\operatorname{dist}(z^{k+1}, Z) \;\leq\; \frac{a\rho}{\sqrt{1 + a^2\rho^2}}\; \operatorname{dist}(z^k, Z).$$

The modulus a measures how far a point with a small residual can be from Z. It is large when the equilibrium set is flat, that is, when many hands have actions that barely matter.

A fast outer loop needs ρ much smaller than $1/a$. But $\|(M + \rho I)^{-1}\| \approx 1/\rho$ then makes the active set of each subproblem fragile. On Leduc we observe a factor of about 10^{-3} for each step at $\rho = 10^{-5}$. On the full river of the demonstration no value of ρ gave both, and the restriction to the support removes the flat directions instead.